

Compute4Me-style Distributed Deep Learning Fabrics: State of the Art and Research Gaps

1. Problem Setting and Conceptual Framing

The Compute4Me idea can be framed as a dockernative, loosely coupled distributed deep learning fabric where heterogeneous Ubuntu machines can join by running a container, advertise their compute capabilities, and cooperatively execute deep learning workloads orchestrated by a master node.

This places the concept at the intersection of several existing areas: distributed deep learning frameworks (e.g., Ray, Horovod), parameter-server and all-reduce architectures, elastic training systems, volunteer and decentralized deep learning, federated/fog/edge computing, and privacy/security-aware distributed learning.^{[1][2][3][4][5][6]}

Key characteristics implied by Compute4Me:

- **Heterogeneous, opportunistic GPUs and CPUs:** Nodes may have different GPU generations, VRAM sizes, and network bandwidths, and may join/leave dynamically.
- **Container-based node onboarding:** A single `docker run . . .` should be sufficient for a machine to join the cluster.
- **Deep-learning-centric workloads:** Training, hyperparameter search, and large-scale inference rather than generic MapReduce.
- **Master-driven task distribution:** A logical controller decides how to split data/model/trials across nodes based on their capabilities.

The following sections survey what has been achieved in adjacent areas, extract common patterns that have become "solved" or commoditized, and highlight genuine research gaps that align closely with the Compute4Me vision.

2. Classical Distributed Deep Learning Architectures

2.1 Parameter Server and All-reduce

Two canonical architectures underpin most distributed data-parallel deep learning today: parameter server (PS) and all-reduce.^{[5][7]}

- In a parameter-server architecture, a group of server nodes maintains global model parameters, while worker nodes pull current parameters, compute gradients on local data shards, and push updates back to the servers for aggregation.^{[7][5]}
- In all-reduce architectures, there is no separate PS tier; instead, workers directly exchange gradients (or parameters) in a collective communication pattern (e.g., ring all-reduce), which can be more bandwidth-efficient and avoids a central bottleneck.^{[2][8]}

These design patterns are now considered standard; both are supported in various forms by TensorFlow, MXNet, PyTorch DDP, Horovod, and system-level libraries like NCCL.^{[8][9][2]}

What is largely solved/common knowledge:

- How to maintain model consistency under synchronous data-parallel training.^{[2][7]}
- How to implement efficient all-reduce schemes (ring, tree, hierarchical).^{[8][2]}
- How to architect PS-based training for large models and datasets in cloud data centers.^{[10][5]}

What remains challenging:

- Scaling to extremely large models (hundreds of billions of parameters) under strict latency and network constraints.^[^10]
- Handling highly heterogeneous nodes (differing GPU capabilities, intermittent connectivity) in a PS/all-reduce regime without sacrificing efficiency.^{[3][10]}

2.2 Representative Systems

Several notable systems embody these ideas:

- **GeePS:** A GPU-specialized parameter server that stores model parameters in GPU memory and optimizes communication to minimize stalls; it demonstrated that a small GPU cluster can outperform large CPU-only clusters for deep learning workloads.^[^11]
- **Horovod:** Uber's library that simplifies distributed deep learning for TensorFlow, PyTorch, Keras, and MXNet using ring all-reduce, enabling near-linear scaling across hundreds of GPUs with minimal changes to user code.^{[2][8]}
- **HPC frameworks:** Traditional HPC stacks (MPI, NCCL, SLURM) underpin many production DDP setups but require significant operational expertise and static cluster assumptions.^{[12][10]}

These systems focus on data-center or HPC environments with relatively homogeneous hardware, tightly managed clusters, and stable connectivity, which differs from the opportunistic, BYO-node philosophy of Compute4Me.

3. Ray and General-purpose AI Compute Engines

3.1 Ray Core and Ray Train

Ray is a general-purpose distributed framework designed for emerging AI applications that mixes task-parallel and actor-based computation under a single dynamic execution engine.^{[13][1]}

Key aspects:

- **Unified abstraction** for remote tasks and stateful actors, enabling reinforcement learning, hyperparameter tuning, and serving workloads in the same system.^{[1][13]}

- **Distributed scheduler and object store:** Ray implements a fully distributed scheduler for high task throughput and a fault-tolerant object store for sharing data across workers. [13][1]
- **High throughput:** Experiments show Ray can schedule over 1.8 million tasks per second and outperform specialized systems in some RL scenarios. [1][13]

On top of Ray core, Ray Train provides an abstraction layer specifically for distributed deep learning, integrating PyTorch and TensorFlow training loops, data ingestion, sharding, checkpointing, and gradient synchronization. [14][15]

- Ray Train allows scaling from single-GPU to multi-node multi-GPU training with minimal code changes, leveraging Ray's scheduling and Ray Data for distributed data loading. [15]
- Recent iterations emphasize bridging the gap between prototype and production by providing a unified, cloud-agnostic environment for distributed training and tuning. [14]

3.2 Relevance to Compute4Me

Ray and Ray Train come closest to the Compute4Me goal of a general-purpose AI compute engine that can orchestrate heterogeneous nodes and workloads.

However, Ray typically assumes an administratively controlled cluster (on-prem or cloud) rather than an open, ad-hoc pool of nodes that can join experimentally via a simple Docker run.

Some Ray deployments do use Docker and Kubernetes, but node joining is usually managed by cluster managers rather than by end users volunteering compute. [15][14]

Common knowledge/solved parts:

- Dynamic task scheduling and actor management for AI workloads in homogeneous clusters. [13][1]
- Integration of distributed training with data ingestion and experiment management. [14][15]

Gaps relative to Compute4Me:

- First-class support for ad-hoc volunteer nodes with variable reliability.
- Built-in mechanisms to automatically infer and exploit node capabilities (e.g., GPU VRAM, bandwidth) for task partitioning, rather than expecting a mostly uniform cluster.

4. Elastic and Heterogeneous Distributed Training

4.1 Elastic Training: Changing GPU Count Mid-flight

A major trend relevant to Compute4Me is elastic training, where the number of GPUs or nodes can grow/shrink while a training job is running.

EasyScale is an elastic training system that maintains accuracy consistency when scaling distributed data-parallel training across heterogeneous GPUs.[^3]

- It preserves the semantics of regular data-parallel training by carefully decoupling training logic from resource allocation and tracing the factors that affect consistency (e.g., batch size, learning rate, gradient scaling).[^3]
- EasyScale introduces abstractions to support fast context switching and dynamic worker assignment, enabling use of idle GPUs opportunistically while guaranteeing deterministic, accuracy-consistent results across resource changes.[^3]
- Deployed in an online serving cluster, it improved overall GPU utilization by over 60%, demonstrating the value of elastic training in production.[^3]

Books and practitioner guides on distributed machine learning similarly highlight **adaptive model training**, where the GPU count changes during a run, and discuss the tight coupling between resource configuration and hyperparameters (batch size, learning rate, etc.).[16][17]

4.2 Resource Allocation and Workload Scheduling

The survey “Resource Allocation and Workload Scheduling for Large-Scale Distributed Deep Learning” comprehensively reviews strategies for allocating resources and scheduling workloads in large-scale DL systems from 2019–2024.[18][10]

- It examines different resource types (GPUs, CPUs, network bandwidth) and scheduling granularities (job-level, pipeline-level, network-flow-level) and their performance goals (throughput, cost, latency, SLAs).[18][10]
- The survey highlights challenges such as heterogeneity in hardware and workloads, fault tolerance, and optimization complexity in data-center-scale training systems.[^10]
- It includes a case study on training large language models to illustrate practical large-scale resource allocation in real distributed DL scenarios.[^10]

4.3 Research Gaps Relevant to Compute4Me

While elastic training and scheduling in data centers are actively studied, there remain gaps for opportunistic, heterogeneous environments like Compute4Me:

- **Fine-grained heterogeneity-aware partitioning:** Most systems assume data-center clusters with relative homogeneity; limited work targets highly heterogeneous GPU capabilities and consumer-grade hardware under dynamic availability.[10][3]
- **Accuracy guarantees under aggressive elasticity:** EasyScale addresses consistency, but general methodologies for arbitrary DL architectures (beyond common vision/NLP models) remain underexplored.[^3]
- **Joint optimization of scheduling and DL hyperparameters** for mixed workloads (training, inference, tuning) on ad-hoc clusters.

These are promising directions if Compute4Me aims to be smarter than existing frameworks about mapping workloads to arbitrarily diverse nodes.

5. Decentralized and Volunteer-based Distributed Deep Learning

5.1 Volunteer Computing Roots

The idea of pooling computational resources from volunteers has precedents in BOINC, Folding@home, and similar scientific computing projects, where volunteers donate CPU/GPU time to large-scale simulations and experiments.^{[19][20]}

These systems demonstrate that large-scale volunteer compute can exceed supercomputers in aggregate FLOPs, but they generally deal with loosely coupled scientific simulations rather than tightly synchronized neural network training.

5.2 Secure Distributed Training at Scale

Recent work explicitly tackles secure distributed training using pooled resources from independent parties.

“Secure Distributed Training at Scale” proposes a Byzantine-tolerant decentralized training protocol that emphasizes communication efficiency.^{[20][19]}

- The authors note that pooling resources from independent groups or volunteers can solve hard DL problems but introduces the risk that any participant can poison or destabilize training.^{[19][20]}
- They design protocols that tolerate Byzantine and Sybil attacks while maintaining scalability, providing theoretical bounds and empirical evaluations on image and language tasks.^{[20][19]}

This line of work is directly relevant if Compute4Me nodes include untrusted or semi-trusted participants.

5.3 Hivemind and Decentralized Deep Learning

Hivemind is a PyTorch library designed for decentralized deep learning over the Internet, specifically to train large models on hundreds of volunteer machines.^{[21][6][^22]}

Key features:

- **Decentralized parameter averaging:** Hivemind uses peer-to-peer protocols and decentralized parameter averaging to avoid a central master node, allowing workers to train even when some peers are slow or unresponsive.^{[6][22]}
- **Decentralized Mixture-of-Experts (MoE):** It supports distributing parts of layers across participants, enabling training of models larger than any single participant’s memory.^[^6]
- **Fault tolerance:** Backpropagation is designed to succeed as long as enough peers respond, making it robust to churn in volunteer networks.^{[22][6]}

Hivemind explicitly targets scenarios very similar to Compute4Me’s vision of “training on thousands of volunteers” but favors a fully decentralized, masterless architecture rather than a central master worker.

5.4 Research Gaps in Decentralized/Volunteer DL

Despite recent advances, several gaps remain:

- **Master-centric variants:** Most decentralized systems avoid a central master; there is less exploration of hybrid architectures where a logical master node orchestrates tasks on a peer-to-peer substrate.
- **Model-agnostic orchestration:** Existing decentralized frameworks typically require specific integration with model code; more generic container-based orchestration with minimal user modification is less explored.^{[22][6]}
- **Security-efficiency trade-offs:** Secure distributed training introduces communication and computation overhead; optimal trade-offs for ad-hoc, non-enterprise clusters remain open.^{[4][20]}

Compute4Me can position itself as an exploration of hybrid architectures that combine master-driven task decomposition with decentralized, fault-tolerant execution among semi-trusted volunteers.

6. Federated, Edge, and Fog-based Training Paradigms

6.1 Federated Learning Systems

Federated learning (FL) focuses on training models across multiple clients while keeping data local for privacy, which overlaps partially with Compute4Me when nodes are independently owned.^{[23][4]}

A recent survey on federated learning systems (FLSs) reviews their system designs and challenges.^[^23]

- It defines FLSs and categorizes them along dimensions such as data distribution, model type, privacy mechanism, communication architecture, federation scale, and motivation (cross-device vs. cross-silo).^[^23]
- It highlights issues in effectiveness, efficiency, and privacy, such as handling non-iid data, communication bottlenecks, and implementing privacy-preserving techniques (e.g., secure aggregation, differential privacy).^[^23]

6.2 Cloud, Edge, Fog, and IoT-based DL

Surveys on deep learning for cloud, edge, fog, and IoT computing discuss deploying and sometimes training models near data sources rather than centralized clouds.^{[24][25]}

- They describe how edge/fog/mist/cloudlet paradigms shift compute to the network edge and outline the implications for latency, bandwidth, and privacy.^{[25][24]}
- Recent surveys emphasize the convergence of ML with fog/edge computing, including distributed training on edge devices and collaborative inference.^{[24][25]}

6.3 Private and Secure Distributed Deep Learning

The survey “Private and Secure Distributed Deep Learning” structures the field into centralized vs. decentralized paradigms, covering techniques for secure training and private inference.[⁴]

- It reviews existing methods for privacy and security in distributed learning (including FL), and identifies open issues spanning both system design and algorithmic defenses.[⁴]
- Challenges include balancing privacy with efficiency, scaling secure aggregation, and defending against data reconstruction or poisoning attacks in distributed setups.[⁴]

6.4 Relevance and Gaps for Compute4Me

Federated, edge, and private DL literature is highly relevant if Compute4Me must:

- Respect data locality and privacy constraints when nodes belong to different organizations.
- Operate over low-bandwidth or intermittent connections typical of edge devices.
- Offer security guarantees in the presence of potentially malicious nodes.

However, most FL/edge systems assume **model-orchestrated updates with limited container abstraction** and focus on data privacy rather than on maximizing utilization of ad-hoc GPU clusters for generic deep learning workloads.[²⁴][²³][⁴]

Gaps include:

- **General-purpose containerized orchestration** of FL-like training across heterogeneous nodes.
- **Integration of privacy mechanisms** into volunteer-based or master-driven deep learning fabrics.

7. Serverless and Cloud-native Distributed ML

Cloud platforms increasingly offer distributed ML capabilities on serverless or managed clusters.

- Databricks recently introduced distributed ML on both serverless and standard clusters, supporting Spark MLlib, Optuna, MLflow, and joblib-based distribution to scale ML workloads without dedicated clusters.[²⁶]
- AWS Lambda and similar FaaS products are used for serverless ML, particularly for inference and lightweight training, emphasizing autoscaling, pay-per-use, and minimal operations.[²⁷][²⁸]

These approaches influence Compute4Me’s design space by showing how far cloud-native abstractions can abstract away cluster management, but they rely heavily on managed services and do not directly address open, user-contributed nodes.

Common knowledge:

- How to deploy models and sometimes distribute ML workloads over managed, autoscaling infrastructure.[²⁸][26]

Gaps relative to Compute4Me:

- Little work on serverless-like orchestration over user-provided, containerized nodes in non-cloud environments.

8. Containerization and DL Pipelines

Most modern ML systems use Docker or other container technologies for packaging training and serving pipelines.

- Practitioner guides and tutorials show how to containerize deep learning applications (e.g., TensorFlow models with Flask, uWSGI, Nginx) to ensure reproducible environments and easier scaling.[^{^29}]

However, containerization is treated largely as a deployment detail; research rarely focuses on container-level orchestration semantics specific to distributed DL beyond using containers as unit-of-deployment in Kubernetes, Docker Swarm, or cloud services.

For Compute4Me, Docker is central to the UX: joining the compute fabric should be as simple as running a prebuilt container, which necessitates a finer-grained look at container-aware scheduling, capability discovery, and isolation tailored to DL workloads.

9. Synthesis: What Is Common Knowledge vs. Open Research

9.1 Widely Solved / Commoditized Areas

The following aspects are well-studied and largely solved, with mature frameworks:

- **Intra-job data-parallel training** on homogeneous or mildly heterogeneous clusters using PS or all-reduce (Horovod, PyTorch DDP, TensorFlow's strategies).[⁷][8][^{^2}]
- **High-performance communication primitives** (all-reduce, broadcast, scatter/gather) via NCCL/MPI and cluster-aware topology optimizations.[¹¹][8][^{^2}]
- **Cluster-centric scheduling and resource allocation** in managed data centers and clouds, including job-level scheduling, basic gang scheduling, and scaling strategies for large DL jobs.[^{^10}]
- **Integrating distributed training into broader ML platforms** (Ray Train, Databricks ML, Horovod on Spark) and providing abstractions that hide low-level details from practitioners.[²⁶][15][^{^14}]
- **Container-based deployment** of ML workloads in managed environments, including Docker/Kubernetes patterns.[^{^29}]

9.2 Emerging but Active Areas (Partially Addressed)

These topics are under active research, with promising but not fully generalized solutions:

- **Elastic training on heterogeneous GPUs:** Systems like EasyScale provide strong results but focus on specific training patterns and cluster types.^[3]
- **Large-scale resource allocation and workload scheduling** for LLMs and large models, especially with mixed workloads (training, inference, tuning) and multi-tenant clusters.^[10]
- **Decentralized volunteer-based training:** Hivemind and secure distributed training protocols show that large models can be trained across volunteers, but these solutions are specialized and still evolving.^{[6][20][22]}
- **Privacy- and security-preserving distributed DL:** Surveys identify many techniques and open challenges; efficient, scalable implementations for general-purpose systems are still being developed.^{[23][4]}
- **Federated learning systems:** Numerous frameworks exist, but they are largely tuned for specific federated paradigms (cross-device or cross-silo) and not for open, general-purpose compute fabrics.^{[24][23]}

9.3 Clear Research Gaps Aligned with Compute4Me

From the synthesis above, several concrete research gaps emerge that map well to your Compute4Me concept.

Gap 1: Heterogeneity- and Capability-aware Orchestration for Volunteer-style DL Fabrics

Most work on resource allocation and scheduling assumes data-center clusters with controlled heterogeneity and stable connectivity.^{[3][10]}

Volunteer-based systems (Hivemind) emphasize decentralization and robustness but less on central, capability-aware task orchestration.

Potential research direction:

- **Design a master-driven orchestration layer** that automatically profiles each Dockerized node (GPU model, VRAM, FLOPS, bandwidth, historical reliability) and uses this to:
 - Decide on data shard sizes, number of concurrent trials, or model partitions per node.
 - Adjust training hyperparameters (batch size, gradient accumulation steps) per node while maintaining overall convergence guarantees.
- **Evaluate schedulers** that treat volunteers as a pool of heterogeneous GPUs and CPUs with stochastic availability, comparing against existing elastic training and decentralized approaches.

Gap 2: Hybrid Master-Decentralized Architectures for Secure Volunteer Training

Existing decentralized training frameworks avoid central masters (Hivemind) or rely on trusted servers for secure aggregation.^{[20][6][^4]}

Compute4Me's design can explore hybrid architectures:

- A logical master node orchestrates high-level tasks, but training updates are aggregated in a Byzantine-tolerant, partially decentralized fashion to avoid single points of failure or trust.
- Investigate combinations of: centrally planned data/model sharding, P2P update exchange, and cryptographic or statistical defenses (Byzantine-resilient aggregation) adapted to deep learning.

This could bridge secure distributed training protocols with practical, Docker-based heterogeneous clusters.

Gap 3: Container-centric, Model-agnostic Execution Interfaces

Most distributed training frameworks require direct integration with model code (modifying the training loop, importing specific libraries), whereas Compute4Me envisions a container as the unit of execution, with minimal assumptions about the internal code beyond a simple API contract.

Possible contributions:

- Define a standardized container interface for DL jobs: environment variables, RPC endpoints, or small sidecar APIs that allow the master to instruct containers (e.g., "train on shard X with config Y"), collect metrics, and handle checkpoints.
- Develop mechanisms for introspecting container capabilities (e.g., through NVIDIA's device query, network benchmarking) in a way that is robust and portable across arbitrary Ubuntu hosts.

There is limited research that treats DL workloads as black-box containers in a volunteer or edge context; most work is in industrial container orchestration (Kubernetes) without DL-specific semantics.^{[29][26]}

Gap 4: Cross-paradigm Scheduling: Training, Inference, and HPO on the Same Ad-hoc Fabric

Existing systems often specialize:

- Ray Train focuses on training and tuning.^{[15][14]}
- Databricks' distributed ML targets Spark-based training and tuning in managed clusters.^[^26]
- Hivemind focuses on training large models.

A Compute4Me fabric could be designed to multiplex different deep learning workloads on the same volunteer cluster:

- Long-running training jobs (e.g., your InceptionResNet exoplanet detection models).
- Batch or streaming inference tasks.
- Hyperparameter and architecture search (e.g., Optuna, Ray Tune-like workloads).

Research questions:

- How to schedule heterogeneous workloads to maximize utilization and fairness while respecting SLAs for latency-sensitive inference tasks.
- How to share models, weights, and intermediate artifacts efficiently across jobs.

Gap 5: Empirical Characterization of Volunteer-based DL Fabrics

There is relatively little systematic measurement of how volunteer-style DL fabrics perform across real-world networks, hardware, and workloads compared to standard clusters.

Possible research contributions:

- Build a prototype Compute4Me system and run large-scale experiments across geographically distributed volunteers.
- Quantify trade-offs in throughput, time-to-accuracy, robustness to churn, and sensitivity to malicious nodes.
- Compare with baselines: Ray on managed clusters, Hivemind-style decentralized training, and EasyScale-like elastic training in controlled environments.^{[6][20][^3]}

Such a study would provide empirical evidence about the practicality of your concept.

Gap 6: Integration of Privacy and Security Mechanisms into Containerized, Volunteer DL Fabrics

Privacy and security surveys highlight many algorithmic techniques (secure aggregation, differential privacy, homomorphic encryption) but less on engineering them into container-based, volunteer DL fabrics.^{[4][23]}

Potential research lines:

- Design privacy-preserving protocols tailored to containerized volunteer nodes, where the master might not fully trust participants' behavior or local data handling.
- Integrate Byzantine-robust aggregation and anomaly detection into the master's orchestration logic, leveraging both cryptographic and statistical defenses.

10. How This Maps to a Research Agenda for Compute4Me

Based on the literature, Compute4Me can be positioned not as yet another DDP wrapper, but as a research platform exploring the system-level and algorithmic questions above.

A plausible multi-stage research program could be:

1. Phase 1 – Core Fabric Prototype

- Implement a Docker-based worker daemon and a master orchestrator that:
 - Discovers node capabilities.
 - Schedules data-parallel training, inference, and HPO tasks.
- Use existing frameworks internally (e.g., PyTorch DDP, Ray, or Horovod) as backends to focus on orchestration rather than reinventing gradient sync.

2. Phase 2 – Heterogeneity- and Elasticity-aware Scheduling

- Implement capability-aware and elastic scheduling algorithms inspired by resource allocation surveys and EasyScale.^{[10][3]}
- Evaluate under realistic workloads (e.g., exoplanet detection models) across varied GPUs.

3. Phase 3 – Security and Privacy Extensions

- Integrate secure distributed training protocols and privacy-preserving techniques appropriate for semi-trusted volunteers.^{[20][4]}
- Study trade-offs in performance and robustness.

4. Phase 4 – Empirical Study of Volunteer-style DL

- Run deployments across collaborators' machines or public volunteers.
- Publish characterization of performance, reliability, and user experience compared to conventional cluster setups.

Such an agenda would be grounded in existing literature while clearly addressing gaps unserved by current distributed DL systems.

11. Conclusion

There is a rich body of work on distributed deep learning in data centers, elastic training, decentralized and volunteer-based learning, federated and edge computing, and privacy/security for distributed ML.

Frameworks like Ray, Horovod, Hivemind, and EasyScale solve many individual pieces of the puzzle but do not fully address the specific niche your Compute4Me idea targets: a Docker-first, capability-aware, potentially volunteer-based deep learning fabric with a master-driven orchestration model.

The main research gaps lie in resource- and capability-aware orchestration over highly heterogeneous volunteer hardware, hybrid centralized–decentralized architectures with security guarantees, container-native execution interfaces for generic DL workloads, cross-paradigm scheduling of training/inference/HPO on ad-hoc clusters, empirical characterization of such fabrics, and integrated privacy/security mechanisms in containerized volunteer settings.

These gaps point to a fertile research space where a carefully designed Compute4Me prototype and accompanying studies could make genuinely novel and impactful contributions.

References

1. [Ray: A Distributed Framework for Emerging AI Applications](#) - The next generation of AI applications will continuously interact with the environment and learn fro...
2. [Horovod: fast and easy distributed deep learning in TensorFlow - ar5iv](#) - The mpirun command distributes `train.py` to four nodes and runs it on four GPUs per node. Horovod can...
3. [EasyScale: Accuracy-consistent Elastic Training for Deep Learning](#) - We introduce EasyScale, an elastic framework that scales distributed training on heterogeneous GPUs ...
4. [Private and Secure Distributed Deep Learning: A Survey](#) - Early online date, 9 Dec 2024 ... Bal, H, Belloum, A, Gommans, L, Van Halteren, A & Klous, S 2025, '...
5. [Parameter Server - an overview | ScienceDirect Topics](#) - A Parameter Server (PS) is an approach developed to scale distributed Machine Learning (ML) contexts...
6. [Hivemind download | SourceForge.net](#) - Hivemind is a PyTorch library for decentralized deep learning across the Internet. Its intended usag...
7. [Parameter Servers and AllReduce - Introduction | Course Notes](#)
8. [Horovod](#) - Horovod is a distributed deep learning training framework for PyTorch, TensorFlow, Keras and Apache ...
9. [Chapter 2: Parameter Server and All-Reduce](#) - Chapter 2: Parameter Server and All-Reduce As described in Chapter 1, Splitting Input Data, to keep ...
10. [Resource Allocation and Workload Scheduling for Large-Scale ...](#) - To illustrate practical large-scale resource allocation and workload scheduling in real distributed ...
11. [GeePS: Scalable deep learning on distributed GPUs with a GPU ...](#) - GeePS is a parameter server supporting data-parallel model training. In data parallel training, the ...
12. [Machine and Deep Learning Frameworks - HPC Wiki](#) - TensorFlow is a machine learning framework with focus on deep neural networks, supporting CPU and GP...
13. [Ray: A Distributed Framework for Emerging {AI} Applications](#)
14. [Distributed Deep Learning with Ray Train is Now In Beta - Anyscale](#) - Powered by Ray, Anyscale empowers AI builders to run and scale all ML and AI workloads on any cloud ...
15. [distributed_training_with_ray_tutorial.py](#)
16. [Introducing adaptive model training - Packt](#) - Access over 7,500 Programming & Development eBooks and videos to advance your IT skills. Enjoy unlim...
17. [Distributed Machine Learning with Python](#) - Chapter 11: Elastic Model Training and Serving The one big challenge in distributed DNN training is ...
18. [\[Literature Review\] Resource Allocation and Workload Scheduling ...](#) - The paper titled "Resource Allocation and Workload Scheduling for Large-Scale Distributed Deep Learn...
19. [Under review as a conference paper at ICLR 2022](#)
20. [Secure Distributed Training at Scale](#) - Many areas of deep learning benefit from using increasingly larger neural networks trained on public...

21. [Build software better, together](#) - GitHub is where people build software. More than 150 million people use GitHub to discover, fork, an...
22. [hivemind · PyPI](#) - Decentralized deep learning in PyTorch. ... Permalink: learning-at-home/hivemind@1a6ff518bc6075c87fd...
23. [A Survey on Federated Learning Systems: Vision, Hype and Reality ...](#) - As data privacy increasingly becomes a critical societal concern, federated learning has been a hot ...
24. [Deep learning models for cloud, edge, fog, and IoT computing ...](#) - The concept of edge computing was developed to offer cloud computing capabilities at the network edg...
25. [Training Machine Learning models at the Edge: A Survey - arXiv](#) - A union between Machine Learning and Edge Computing, where ML models are deployed at the edge, close...
26. [Announcing the Public Preview of Distributed ML on Serverless and ...](#) - Databricks users can now run distributed ML workloads on both serverless and standard clusters, incl...
27. [Serverless Deep Learning with AWS Lambda - O'Reilly](#) - By the end of the live online course, you'll understand: What is Serverless and AWS Lambda and how t...
28. [Running Serverless ML on AWS Lambda | Better Dev](#) - Yes, you can run Machine Learning models on serverless, directly with AWS Lambda. I know because I b...
29. [How to use Docker containers and Docker Compose for Deep ...](#) - In this article, we will containerize our Deep Learning application using Docker. Our application co...